

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE MANAGEMENT SYSTEM ASSIGNMENT-9

CAB BOOKING MANAGEMENT SYSTEM

Advanced SQL Queries Using Joins and Subqueries

BACHELORS OF COMPUTER APPLICATION- III

NAME: PREKSHA V

REGISTER NUMBER: 2511021240063

SUBMITTING TO,

RENITA JOY R

1. Objective

This assignment develops advanced SQL queries for the Cab Booking Management System using joins and subqueries. The queries combine data from several related tables and are written as meaningful reports for the cab service: booking and payment tracking, driver performance, customer spending, fleet planning and an overall business summary. For every report the SQL query, the actual output and a short explanation are given.

2. Database and Data Used

All reports run on the CabBookingDB database created in Assignment 5, with the integrity constraints of Assignment 6 and the sample data of Assignment 7. The tables and the number of records they contain are shown below.

Table	Records	Key columns used in the reports
CUSTOMER	10	Customer_ID, Name, Phone, Email
DRIVER	8	Driver_ID, Name, Status
CAB	8	Cab_ID, Driver_ID (FK), Vehicle_Number, Cab_Type, Status
BOOKING	15	Booking_ID, Customer_ID, Driver_ID, Cab_ID (FKs), Pickup/Drop_Location, Booking_Status
TRIP	12	Trip_ID, Booking_ID (FK), Distance, Fare, Trip_Status
PAYMENT	10	Payment_ID, Booking_ID (FK), Amount, Payment_Method, Payment_Status
RATING	8	Rating_ID, Booking_ID (FK), Rating
DRIVER_LOCATION	16	Location_ID, Driver_ID (FK), Latitude, Longitude, Updated_Time

The joins follow the foreign keys: CUSTOMER, DRIVER and CAB connect to BOOKING; TRIP, PAYMENT and RATING connect to BOOKING through Booking_ID; and DRIVER_LOCATION connects to DRIVER.

The queries use standard SQL supported by MySQL 8.0. They were executed on a MySQL-compatible server (MariaDB 10.11) loaded with the Assignment 7 data, and the outputs below are the actual results.

Reports at a glance

No.	Report	Technique	Tables used
1	Complete booking report	INNER JOIN of four tables	BOOKING, CUSTOMER, DRIVER, CAB
2	Booking life-cycle report	chained LEFT JOINs	BOOKING, TRIP, PAYMENT, RATING
3	Completed rides that were not rated	LEFT JOIN with IS NULL	BOOKING, CUSTOMER, DRIVER, TRIP, RATING
4	Driver performance scorecard	LEFT JOIN, GROUP BY, aggregates	DRIVER, BOOKING, TRIP, RATING
5	Revenue and fare per kilometre by cab type	four-table JOIN	CAB, BOOKING, TRIP, PAYMENT
6	Customer spending report	LEFT JOIN with a condition	CUSTOMER, BOOKING, PAYMENT
7	Replacement cabs for a cab under maintenance	SELF JOIN	CAB, DRIVER
8	Pickup location demand report	LEFT JOIN, CASE, GROUP BY	BOOKING, TRIP
9	Trips with a fare above the average	scalar subquery in WHERE	TRIP, BOOKING, CUSTOMER, DRIVER
10	Customers who paid by UPI	IN subquery	CUSTOMER, BOOKING, PAYMENT
11	Drivers who have never been rated	NOT EXISTS	DRIVER, BOOKING, RATING
12	Customers with a pending payment	EXISTS	CUSTOMER, BOOKING, PAYMENT
13	Trips above the driver's own average fare	correlated subquery	TRIP, BOOKING, DRIVER
14	Latest known location of available drivers	correlated subquery	DRIVER, DRIVER_LOCATION
15	Revenue share of each cab type	subqueries in FROM	CAB, BOOKING, PAYMENT
16	Customer activity report	subqueries in SELECT	CUSTOMER, BOOKING, PAYMENT
17	Business summary report	scalar subqueries	BOOKING, PAYMENT, RATING

3. Reports Using Joins

A join combines rows from two or more tables using a related column. INNER JOIN returns only rows that have a match in both tables. LEFT JOIN returns all rows of the left table and shows NULL where no match exists. A self join joins a table with itself using two aliases.

Query 1: Complete booking report (INNER JOIN of four tables)

Report: Shows who booked, which driver was assigned and which cab was used for every booking.

SQL QUERY

```
SELECT b.Booking_ID, DATE(b.Booking_Date) AS Booking_Date,
       c.Name AS Customer, d.Name AS Driver,
       cb.Vehicle_Number, cb.Cab_Type, b.Booking_Status
FROM BOOKING b
JOIN CUSTOMER c ON b.Customer_ID = c.Customer_ID
JOIN DRIVER d   ON b.Driver_ID   = d.Driver_ID
JOIN CAB cb     ON b.Cab_ID      = cb.Cab_ID
ORDER BY b.Booking_ID;
```

OUTPUT

```
+------------+--------------+----------------+-----------------+----------------+-----------+----------------+
| Booking_ID | Booking_Date | Customer       | Driver          | Vehicle_Number | Cab_Type  | Booking_Status |
+------------+--------------+----------------+-----------------+----------------+-----------+----------------+
|       5001 | 2026-10-01   | Rahul Sharma   | Venkatesh Rao   | KA04GH9021     | SUV       | Completed      |
|       5002 | 2026-10-02   | Ananya Iyer    | Ramesh Kumar    | KA01AB1234     | Sedan     | Completed      |
|       5003 | 2026-10-03   | Karthik Nair   | Manjunath Gowda | KA03CD7788     | Hatchback | Completed      |
|       5004 | 2026-10-04   | Sneha Patil    | Prakash Shetty  | KA05LM1290     | Sedan     | Completed      |
|       5005 | 2026-10-06   | Arjun Mehta    | Mohammed Irfan  | KA03PQ6643     | SUV       | Completed      |
|       5006 | 2026-10-08   | Divya Krishnan | Ramesh Kumar    | KA01AB1234     | Sedan     | Completed      |
|       5007 | 2026-10-10   | Rohan Das      | Venkatesh Rao   | KA04GH9021     | SUV       | Completed      |
|       5008 | 2026-10-12   | Meera Joshi    | Manjunath Gowda | KA03CD7788     | Hatchback | Completed      |
|       5009 | 2026-10-15   | Vikram Singh   | Prakash Shetty  | KA05LM1290     | Sedan     | Completed      |
|       5010 | 2026-10-17   | Pooja Reddy    | Mohammed Irfan  | KA03PQ6643     | SUV       | Completed      |
|       5011 | 2026-10-19   | Rahul Sharma   | Manjunath Gowda | KA03CD7788     | Hatchback | Cancelled      |
|       5012 | 2026-10-20   | Sneha Patil    | Ramesh Kumar    | KA01AB1234     | Sedan     | Cancelled      |
|       5013 | 2026-10-23   | Ananya Iyer    | Suresh Babu     | KA05MN4521     | SUV       | Confirmed      |
|       5014 | 2026-10-23   | Arjun Mehta    | Anil Reddy      | KA01JK5567     | Hatchback | Confirmed      |
|       5015 | 2026-10-24   | Rohan Das      | Venkatesh Rao   | KA04GH9021     | SUV       | Pending        |
+------------+--------------+----------------+-----------------+----------------+-----------+----------------+
```

Explanation: Each INNER JOIN follows a foreign key (BOOKING to CUSTOMER, DRIVER and CAB) and keeps only rows that have a match. Because every booking has a customer, driver and cab, all 15 bookings appear with readable names instead of IDs.

Query 2: Booking life-cycle report (chained LEFT JOINs)

Report: Tracks every booking through trip, payment and rating, even when a stage has not happened yet.

SQL QUERY

```
SELECT b.Booking_ID, b.Booking_Status, t.Trip_Status, t.Fare,
       p.Payment_Status, r.Rating
FROM BOOKING b
LEFT JOIN TRIP t    ON t.Booking_ID = b.Booking_ID
LEFT JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
LEFT JOIN RATING r  ON r.Booking_ID = b.Booking_ID
ORDER BY b.Booking_ID;
```

OUTPUT

```
+------------+----------------+-------------+--------+----------------+--------+
| Booking_ID | Booking_Status | Trip_Status | Fare   | Payment_Status | Rating |
+------------+----------------+-------------+--------+----------------+--------+
|       5001 | Completed      | Completed   | 350.00 | Successful     |      5 |
|       5002 | Completed      | Completed   | 780.00 | Successful     |      4 |
|       5003 | Completed      | Completed   | 520.00 | Successful     |      5 |
|       5004 | Completed      | Completed   | 210.00 | Successful     |      3 |
|       5005 | Completed      | Completed   | 290.00 | Successful     |      4 |
|       5006 | Completed      | Completed   | 180.00 | Successful     |      5 |
|       5007 | Completed      | Completed   | 420.00 | Successful     |      4 |
|       5008 | Completed      | Completed   | 260.00 | Successful     |   NULL |
|       5009 | Completed      | Completed   | 950.00 | Successful     |      5 |
|       5010 | Completed      | Completed   | 360.00 | Pending        |   NULL |
|       5011 | Cancelled      | NULL        |   NULL | NULL           |   NULL |
|       5012 | Cancelled      | Cancelled   |   0.00 | NULL           |   NULL |
|       5013 | Confirmed      | Started     |   NULL | NULL           |   NULL |
|       5014 | Confirmed      | NULL        |   NULL | NULL           |   NULL |
|       5015 | Pending        | NULL        |   NULL | NULL           |   NULL |
+------------+----------------+-------------+--------+----------------+--------+
```

Explanation: LEFT JOIN keeps all bookings and shows NULL where the related record does not exist. Cancelled booking 5011 and the pending booking 5015 have no trip or payment, and several completed rides have no rating. An INNER JOIN would have hidden these rows.

Query 3: Completed rides that were not rated (LEFT JOIN with IS NULL)

Report: Finds customers who could be asked for feedback.

SQL QUERY

```
SELECT b.Booking_ID, c.Name AS Customer, d.Name AS Driver,
       b.Drop_Location, t.Fare
FROM BOOKING b
JOIN CUSTOMER c ON b.Customer_ID = c.Customer_ID
JOIN DRIVER d   ON b.Driver_ID   = d.Driver_ID
JOIN TRIP t     ON t.Booking_ID  = b.Booking_ID
LEFT JOIN RATING r ON r.Booking_ID = b.Booking_ID
WHERE b.Booking_Status = 'Completed'
  AND r.Rating_ID IS NULL;
```

OUTPUT

```
+------------+-------------+-----------------+--------------------+--------+
| Booking_ID | Customer    | Driver          | Drop_Location      | Fare   |
+------------+-------------+-----------------+--------------------+--------+
|       5008 | Meera Joshi | Manjunath Gowda | Majestic Bus Stand | 260.00 |
|       5010 | Pooja Reddy | Mohammed Irfan  | Indiranagar        | 360.00 |
+------------+-------------+-----------------+--------------------+--------+
```

Explanation: After the LEFT JOIN, rows where Rating_ID is NULL are bookings without a rating (an anti-join). Bookings 5008 and 5010 are completed but unrated, which matches the optional 1:0..1 Booking-Rating relationship.

Query 4: Driver performance scorecard (LEFT JOIN, GROUP BY, aggregates)

Report: Summarises completed trips, distance, earnings and average rating for every driver.

SQL QUERY

```
SELECT d.Driver_ID, d.Name AS Driver, d.Status,
       COUNT(DISTINCT b.Booking_ID)   AS Completed_Trips,
       COALESCE(SUM(t.Distance), 0)   AS Total_Distance_Km,
       COALESCE(SUM(t.Fare), 0)       AS Total_Fare,
       ROUND(AVG(r.Rating), 2)        AS Avg_Rating
FROM DRIVER d
LEFT JOIN BOOKING b ON b.Driver_ID = d.Driver_ID
                   AND b.Booking_Status = 'Completed'
LEFT JOIN TRIP t    ON t.Booking_ID = b.Booking_ID
LEFT JOIN RATING r  ON r.Booking_ID = b.Booking_ID
GROUP BY d.Driver_ID, d.Name, d.Status
ORDER BY Total_Fare DESC;
```

OUTPUT

Driver_ID	Driver	Status	Completed_Trips	Total_Distance_Km	Total_Fare	Avg_Rating
207	Prakash Shetty	Available	2	50.90	1160.00	4.00
201	Ramesh Kumar	Available	2	44.30	960.00	4.50
203	Manjunath Gowda	Available	2	35.50	780.00	5.00
205	Venkatesh Rao	Available	2	39.50	770.00	4.50
208	Mohammed Irfan	Available	2	32.50	650.00	4.00
202	Suresh Babu	Busy	0	0.00	0.00	NULL
204	Imran Khan	Offline	0	0.00	0.00	NULL
206	Anil Reddy	Busy	0	0.00	0.00	NULL

Explanation: The condition on Booking_Status is placed in the ON clause so that drivers without completed trips are still listed. COALESCE replaces NULL sums with 0. Prakash Shetty earned the most (1,160); drivers 202, 204 and 206 have no completed trips and no ratings, so their average rating is NULL.

Query 5: Revenue and fare per kilometre by cab type (four-table JOIN)

Report: Compares the earning power of Sedan, SUV and Hatchback cabs.

SQL QUERY

```
SELECT cb.Cab_Type,
       COUNT(*)                          AS Paid_Trips,
       SUM(t.Distance)                   AS Total_Distance_Km,
       SUM(p.Amount)                     AS Revenue,
       ROUND(SUM(t.Fare) / SUM(t.Distance), 2) AS Fare_Per_Km
FROM CAB cb
JOIN BOOKING b ON b.Cab_ID     = cb.Cab_ID
JOIN TRIP t    ON t.Booking_ID = b.Booking_ID
JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
WHERE p.Payment_Status = 'Successful'
GROUP BY cb.Cab_Type
ORDER BY Revenue DESC;
```

OUTPUT

```
+-----------+------------+-------------------+---------+-------------+
| Cab_Type  | Paid_Trips | Total_Distance_Km | Revenue | Fare_Per_Km |
+-----------+------------+-------------------+---------+-------------+
| Sedan     |          4 |             95.20 | 2120.00 |       22.27 |
| SUV       |          3 |             54.10 | 1060.00 |       19.59 |
| Hatchback |          2 |             35.50 |  780.00 |       21.97 |
+-----------+------------+-------------------+---------+-------------+
```

Explanation: Only trips with a successful payment are counted, so the pending payment of booking 5010 is excluded. Sedans bring the highest revenue (2,120 from 4 trips) and SUVs the lowest fare per kilometre, which helps in deciding how many cabs of each type to keep.

Query 6: Customer spending report (LEFT JOIN with a condition)

Report: Ranks customers by the amount they have actually paid.

SQL QUERY

```
SELECT c.Customer_ID, c.Name AS Customer,
       COUNT(DISTINCT b.Booking_ID)  AS Bookings,
       COUNT(p.Payment_ID)           AS Successful_Payments,
       COALESCE(SUM(p.Amount), 0)    AS Total_Spent
FROM CUSTOMER c
LEFT JOIN BOOKING b ON b.Customer_ID = c.Customer_ID
LEFT JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
                   AND p.Payment_Status = 'Successful'
GROUP BY c.Customer_ID, c.Name
ORDER BY Total_Spent DESC;
```

OUTPUT

```
+-------------+----------------+----------+---------------------+-------------+
| Customer_ID | Customer       | Bookings | Successful_Payments | Total_Spent |
+-------------+----------------+----------+---------------------+-------------+
|         109 | Vikram Singh   |        1 |                   1 |      950.00 |
|         102 | Ananya Iyer    |        2 |                   1 |      780.00 |
|         103 | Karthik Nair   |        1 |                   1 |      520.00 |
|         107 | Rohan Das      |        2 |                   1 |      420.00 |
|         101 | Rahul Sharma   |        2 |                   1 |      350.00 |
|         105 | Arjun Mehta    |        2 |                   1 |      290.00 |
|         108 | Meera Joshi    |        1 |                   1 |      260.00 |
|         104 | Sneha Patil    |        2 |                   1 |      210.00 |
|         106 | Divya Krishnan |        1 |                   1 |      180.00 |
|         110 | Pooja Reddy    |        1 |                   0 |        0.00 |
+-------------+----------------+----------+---------------------+-------------+
```

Explanation: Vikram Singh is the top spender (950). Pooja Reddy has a booking but has paid nothing yet because her payment is Pending, so she appears with 0 instead of disappearing from the report.

Query 7: Replacement cabs for a cab under maintenance (SELF JOIN)

Report: Suggests available cabs of the same type when a cab is out of service.

SQL QUERY

```
SELECT m.Cab_ID AS Cab_In_Maintenance, m.Cab_Type,
       r.Cab_ID AS Replacement_Cab, r.Vehicle_Number,
       r.Model, d.Name AS Driver
FROM CAB m
JOIN CAB r    ON r.Cab_Type = m.Cab_Type
             AND r.Cab_ID  <> m.Cab_ID
             AND r.Status   = 'Available'
JOIN DRIVER d ON d.Driver_ID = r.Driver_ID
WHERE m.Status = 'Maintenance';
```

OUTPUT

```
+--------------------+----------+-----------------+----------------+--------------------+----------------+
| Cab_In_Maintenance | Cab_Type | Replacement_Cab | Vehicle_Number | Model              | Driver         |
+--------------------+----------+-----------------+----------------+--------------------+----------------+
|                304 | Sedan    |             301 | KA01AB1234     | Maruti Swift Dzire | Ramesh Kumar   |
|                304 | Sedan    |             307 | KA05LM1290     | Honda Amaze        | Prakash Shetty |
+--------------------+----------+-----------------+----------------+--------------------+----------------+
```

Explanation: The CAB table is joined with itself using two aliases: m for the cab in maintenance and r for candidate replacements of the same type. Cab 304 (Sedan) can be replaced by sedans 301 or 307.

Query 8: Pickup location demand report (LEFT JOIN, CASE, GROUP BY)

Report: Shows which pickup areas generate the most bookings and fare.

SQL QUERY

```
SELECT b.Pickup_Location,
       COUNT(*) AS Bookings,
       SUM(CASE WHEN b.Booking_Status = 'Completed'
                THEN 1 ELSE 0 END) AS Completed,
       COALESCE(SUM(t.Fare), 0)    AS Total_Fare
FROM BOOKING b
LEFT JOIN TRIP t ON t.Booking_ID = b.Booking_ID
GROUP BY b.Pickup_Location
ORDER BY Bookings DESC, Total_Fare DESC;
```

OUTPUT

```
+-------------------------+----------+-----------+------------+
| Pickup_Location         | Bookings | Completed | Total_Fare |
+-------------------------+----------+-----------+------------+
| Alliance University     |        3 |         2 |     770.00 |
| Koramangala             |        2 |         1 |     520.00 |
| Anekal Town             |        1 |         1 |     950.00 |
| HSR Layout              |        1 |         1 |     780.00 |
| Whitefield              |        1 |         1 |     360.00 |
| Indiranagar             |        1 |         1 |     290.00 |
| BTM Layout              |        1 |         1 |     260.00 |
| Jayanagar               |        1 |         1 |     210.00 |
| Electronic City Phase 2 |        1 |         1 |     180.00 |
| Marathahalli            |        1 |         0 |       0.00 |
| MG Road                 |        1 |         0 |       0.00 |
| Electronic City Phase 1 |        1 |         0 |       0.00 |
+-------------------------+----------+-----------+------------+
```

Explanation: Alliance University is the busiest pickup point with 3 bookings (2 completed). CASE counts only completed bookings inside the group, and LEFT JOIN keeps pickup points whose bookings have no trip yet.

4. Reports Using Subqueries

A subquery is a query nested inside another query. A scalar subquery returns one value; IN compares with a list of values; EXISTS and NOT EXISTS test whether matching rows are present; a correlated subquery uses a value from the outer row; and a subquery in the FROM clause (derived table) acts as a temporary table.

Query 9: Trips with a fare above the average (scalar subquery in WHERE)

Report: Identifies high-value trips with the customer and driver involved.

SQL QUERY

```
SELECT t.Trip_ID, c.Name AS Customer, d.Name AS Driver,
       t.Distance, t.Fare
FROM TRIP t
JOIN BOOKING b  ON t.Booking_ID  = b.Booking_ID
JOIN CUSTOMER c ON b.Customer_ID = c.Customer_ID
JOIN DRIVER d   ON b.Driver_ID   = d.Driver_ID
WHERE t.Fare > (SELECT AVG(Fare)
                FROM TRIP
                WHERE Trip_Status = 'Completed')
ORDER BY t.Fare DESC;
```

OUTPUT

```
+---------+-------------+-----------------+----------+--------+
| Trip_ID | Customer    | Driver          | Distance | Fare   |
+---------+-------------+-----------------+----------+--------+
|    7009 | Vikram Singh | Prakash Shetty  |    41.50 | 950.00 |
|    7002 | Ananya Iyer  | Ramesh Kumar    |    36.20 | 780.00 |
|    7003 | Karthik Nair | Manjunath Gowda |    22.80 | 520.00 |
+---------+-------------+-----------------+----------+--------+
```

Explanation: The inner query returns one value, the average fare of completed trips (432.00). The outer query keeps only trips above it: 7009, 7002 and 7003. The average is recalculated automatically whenever the data changes.

Query 10: Customers who paid by UPI (IN subquery)

Report: Builds a customer list for UPI-related offers.

SQL QUERY

```
SELECT c.Customer_ID, c.Name, c.Phone
FROM CUSTOMER c
WHERE c.Customer_ID IN (SELECT b.Customer_ID
                        FROM BOOKING b
                        JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
                        WHERE p.Payment_Method = 'UPI');
```

OUTPUT

```
+-------------+-------------+------------+
| Customer_ID | Name        | Phone      |
+-------------+-------------+------------+
|         101 | Rahul Sharma | 9886012101 |
|         103 | Karthik Nair | 9886012103 |
|         105 | Arjun Mehta  | 9886012105 |
|         108 | Meera Joshi  | 9886012108 |
|         110 | Pooja Reddy  | 9886012110 |
+-------------+-------------+------------+
```

Explanation: The subquery returns the list of customer IDs linked to UPI payments, and IN keeps the customers in that list. Five customers have used UPI.

Query 11: Drivers who have never been rated (NOT EXISTS)

Report: Finds drivers who have no customer feedback yet.

SQL QUERY

```
SELECT d.Driver_ID, d.Name, d.Status
FROM DRIVER d
WHERE NOT EXISTS (SELECT 1
                  FROM BOOKING b
                  JOIN RATING r ON r.Booking_ID = b.Booking_ID
                  WHERE b.Driver_ID = d.Driver_ID);
```

OUTPUT

```
+-----------+-------------+---------+
| Driver_ID | Name        | Status  |
+-----------+-------------+---------+
|       202 | Suresh Babu | Busy    |
|       204 | Imran Khan  | Offline |
|       206 | Anil Reddy  | Busy    |
+-----------+-------------+---------+
```

Explanation: For each driver, the subquery looks for a rating on any of that driver's bookings. NOT EXISTS keeps drivers for whom nothing is found: drivers 202, 204 and 206, who match the NULL ratings seen in Query 4.

Query 12: Customers with a pending payment (EXISTS)

Report: Lists customers who must be reminded to complete a payment.

SQL QUERY

```
SELECT c.Customer_ID, c.Name, c.Phone, c.Email
FROM CUSTOMER c
WHERE EXISTS (SELECT 1
              FROM BOOKING b
              JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
              WHERE b.Customer_ID = c.Customer_ID
                AND p.Payment_Status = 'Pending');
```

OUTPUT

```
+-------------+-------------+------------+-----------------------+
| Customer_ID | Name        | Phone      | Email                 |
+-------------+-------------+------------+-----------------------+
|         110 | Pooja Reddy | 9886012110 | pooja.reddy@gmail.com |
+-------------+-------------+------------+-----------------------+
```

Explanation: EXISTS returns true as soon as one matching pending payment is found for the customer, so it is efficient for yes/no checks. Only Pooja Reddy (booking 5010, amount 360) has an outstanding payment.

Query 13: Trips above the driver's own average fare (correlated subquery)

Report: Highlights each driver's best-paying trips.

SQL QUERY

```
SELECT t.Trip_ID, t.Booking_ID, d.Name AS Driver, t.Fare
FROM TRIP t
JOIN BOOKING b ON t.Booking_ID = b.Booking_ID
JOIN DRIVER d  ON b.Driver_ID  = d.Driver_ID
WHERE t.Trip_Status = 'Completed'
  AND t.Fare > (SELECT AVG(t2.Fare)
                FROM TRIP t2
                JOIN BOOKING b2 ON t2.Booking_ID = b2.Booking_ID
                WHERE b2.Driver_ID = b.Driver_ID
                  AND t2.Trip_Status = 'Completed')
ORDER BY d.Name, t.Fare DESC;
```

OUTPUT

```
+---------+------------+-----------------+--------+
| Trip_ID | Booking_ID | Driver          | Fare   |
+---------+------------+-----------------+--------+
|    7003 |       5003 | Manjunath Gowda | 520.00 |
|    7010 |       5010 | Mohammed Irfan  | 360.00 |
|    7009 |       5009 | Prakash Shetty  | 950.00 |
|    7002 |       5002 | Ramesh Kumar    | 780.00 |
|    7007 |       5007 | Venkatesh Rao   | 420.00 |
+---------+------------+-----------------+--------+
```

Explanation: The subquery is correlated: it refers to b.Driver_ID of the outer row, so the average is recalculated for each driver. One trip above each of the five working drivers' averages is returned.

Query 14: Latest known location of available drivers (correlated subquery)

Report: Supports matching a booking with the nearest available driver.

SQL QUERY

```
SELECT d.Driver_ID, d.Name, d.Status,
       l.Latitude, l.Longitude, l.Updated_Time
FROM DRIVER d
JOIN DRIVER_LOCATION l ON l.Driver_ID = d.Driver_ID
WHERE d.Status = 'Available'
  AND l.Updated_Time = (SELECT MAX(l2.Updated_Time)
                        FROM DRIVER_LOCATION l2
                        WHERE l2.Driver_ID = d.Driver_ID);
```

OUTPUT

```
+-----------+-----------------+-----------+-----------+-----------+---------------------+
| Driver_ID | Name            | Status    | Latitude  | Longitude | Updated_Time        |
+-----------+-----------------+-----------+-----------+-----------+---------------------+
|       201 | Ramesh Kumar    | Available | 12.925000 | 77.615000 | 2026-10-24 09:30:00 |
|       203 | Manjunath Gowda | Available | 12.940000 | 77.630000 | 2026-10-24 09:30:00 |
|       205 | Venkatesh Rao   | Available | 12.716000 | 77.710000 | 2026-10-24 09:30:00 |
|       207 | Prakash Shetty  | Available | 12.930000 | 77.599000 | 2026-10-24 09:30:00 |
|       208 | Mohammed Irfan  | Available | 12.981000 | 77.644000 | 2026-10-24 09:30:00 |
+-----------+-----------------+-----------+-----------+-----------+---------------------+
```

Explanation: Every driver has several location records, so the subquery finds the most recent update time for that driver, and only that row is kept. The result is one current position for each of the five available drivers.

Query 15: Revenue share of each cab type (subqueries in FROM)

Report: Shows what percentage of total revenue comes from each cab type.

SQL QUERY

```
SELECT r.Cab_Type, r.Revenue,
       ROUND(r.Revenue * 100 / tot.Total_Revenue, 2) AS Revenue_Share_Percent
FROM (SELECT cb.Cab_Type, SUM(p.Amount) AS Revenue
      FROM CAB cb
      JOIN BOOKING b ON b.Cab_ID = cb.Cab_ID
      JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
      WHERE p.Payment_Status = 'Successful'
      GROUP BY cb.Cab_Type) r
CROSS JOIN (SELECT SUM(Amount) AS Total_Revenue
            FROM PAYMENT
            WHERE Payment_Status = 'Successful') tot
ORDER BY r.Revenue DESC;
```

OUTPUT

```
+-----------+---------+-----------------------+
| Cab_Type  | Revenue | Revenue_Share_Percent |
+-----------+---------+-----------------------+
| Sedan     | 2120.00 |                 53.54 |
| SUV       | 1060.00 |                 26.77 |
| Hatchback |  780.00 |                 19.70 |
+-----------+---------+-----------------------+
```

Explanation: Two derived tables are created in the FROM clause: r holds the revenue of each cab type and tot holds the overall revenue (3,960). Joining them allows the percentage to be calculated in one query. Sedans contribute about 53.5 percent of revenue.

Query 16: Customer activity report (subqueries in SELECT)

Report: Gives a one-line profile of every customer's bookings, cancellations and payments.

SQL QUERY

```
SELECT c.Customer_ID, c.Name,
       (SELECT COUNT(*) FROM BOOKING b
        WHERE b.Customer_ID = c.Customer_ID) AS Total_Bookings,
       (SELECT COUNT(*) FROM BOOKING b
        WHERE b.Customer_ID = c.Customer_ID
          AND b.Booking_Status = 'Cancelled') AS Cancelled,
       (SELECT COALESCE(SUM(p.Amount), 0)
        FROM BOOKING b
        JOIN PAYMENT p ON p.Booking_ID = b.Booking_ID
        WHERE b.Customer_ID = c.Customer_ID
          AND p.Payment_Status = 'Successful') AS Amount_Paid
FROM CUSTOMER c
ORDER BY Amount_Paid DESC;
```

OUTPUT

```
+-------------+---------------+---------------+-----------+-------------+
| Customer_ID | Name          | Total_Bookings | Cancelled | Amount_Paid |
+-------------+---------------+---------------+-----------+-------------+
|         109 | Vikram Singh  |             1 |         0 |      950.00 |
|         102 | Ananya Iyer   |             2 |         0 |      780.00 |
|         103 | Karthik Nair  |             1 |         0 |      520.00 |
|         107 | Rohan Das     |             2 |         0 |      420.00 |
|         101 | Rahul Sharma  |             2 |         1 |      350.00 |
|         105 | Arjun Mehta   |             2 |         0 |      290.00 |
|         108 | Meera Joshi   |             1 |         0 |      260.00 |
|         104 | Sneha Patil   |             2 |         1 |      210.00 |
|         106 | Divya Krishnan |             1 |         0 |      180.00 |
|         110 | Pooja Reddy   |             1 |         0 |        0.00 |
+-------------+---------------+---------------+-----------+-------------+
```

Explanation: Each column is produced by its own correlated scalar subquery, evaluated once per customer. Rahul Sharma and Sneha Patil are the customers who have cancelled a booking.

5. Overall Business Summary

The final report combines several scalar subqueries to present the key figures of the cab service in one row.

Query 17: Business summary report (scalar subqueries)

Report: Presents the key figures of the cab service in a single row.

SQL QUERY

```
SELECT (SELECT COUNT(*) FROM BOOKING) AS Total_Bookings,
       (SELECT COUNT(*) FROM BOOKING
        WHERE Booking_Status = 'Completed') AS Completed,
       (SELECT COUNT(*) FROM BOOKING
        WHERE Booking_Status = 'Cancelled') AS Cancelled,
       ROUND((SELECT COUNT(*) FROM BOOKING
              WHERE Booking_Status = 'Cancelled') * 100
             / (SELECT COUNT(*) FROM BOOKING), 2) AS Cancel_Rate_Percent,
       (SELECT SUM(Amount) FROM PAYMENT
        WHERE Payment_Status = 'Successful') AS Revenue,
       (SELECT SUM(Amount) FROM PAYMENT
        WHERE Payment_Status = 'Pending') AS Pending_Amount,
       (SELECT ROUND(AVG(Rating), 2) FROM RATING) AS Avg_Rating;
```

OUTPUT

```
+----------------+-----------+-----------+--------------------+---------+----------------+------------+
| Total_Bookings | Completed | Cancelled | Cancel_Rate_Percent | Revenue | Pending_Amount | Avg_Rating |
+----------------+-----------+-----------+--------------------+---------+----------------+------------+
|             15 |        10 |         2 |              13.33 | 3960.00 |         360.00 |       4.38 |
+----------------+-----------+-----------+--------------------+---------+----------------+------------+
```

Explanation: Each figure comes from an independent scalar subquery, and the cancellation rate combines two of them. Out of 15 bookings, 10 are completed and 2 cancelled (13.33 percent); revenue received is 3,960, 360 is still pending, and the average rating is 4.38.

6. Conclusion

Seventeen reports were developed using joins and subqueries across the eight tables of the Cab Booking Management System. INNER JOIN, LEFT JOIN and self join combined related records and exposed missing ones, such as unrated rides and drivers without trips. Scalar, IN, EXISTS, NOT EXISTS, correlated and derived-table subqueries answered questions that depend on averages, latest values and the presence or absence of related records. Together, the reports give the cab service useful information about bookings, revenue, driver performance, customer behaviour and fleet availability, and show why the database was designed with separate tables connected by foreign keys.